# Gene Expression Analysis

Ridah Shaista Shanavas

2026-10-22

RESOURCES: https://www.r-bloggers.com/2024/11/the-complete-guide-to-using-setdiff-in-r-examples-and-best-practices/ Also used AI for brainstorming and debugging.

## R Markdown

```
# Load packages
library(tidyverse) # note: tidyverse includes ggplot2

# STEP 1: Loading the data

# Import the original CSV files
covar <- read_csv("GSE85307_covar.csv")
gene_expr <- read_csv("GSE85307_geneExpression.csv")
gene_names <- read_csv("GSE85307_geneNames.csv")
```

### Initial Data Inspection

```
# STEP 2: Inspecting the data

# Checking how many rows and columns are there in each file
dim(covar)      # RESULT: 157 X 10
```

```
## [1] 157  10
```

```
dim(gene_expr)  # RESULT: 500 X 158
```

```
## [1] 500 158
```

```
dim(gene_names) # RESULT: 750 X 2
```

```
## [1] 750   2
```

```
# Checking what the column names are
#names(covar)
#names(gene_expr)  # column names seem to match the geoID in covar
#names(gene_names) # contains a Probe ID and Name....will likely use this probe ID
# to match gene expression with gene name...gene expression columns seem to be
# like probe ID

# Viewing a few rows of each dataset
#head(covar)
#head(gene_expr[, 1:min(6, ncol(gene_expr))])
#head(gene_expr[,1:6]) # looking at first 6 columns
#head(gene_names)

# Viewing each dataset fully
#View(covar)
#View(gene_expr)
#View(gene_names)
```

After inspecting the three datasets using View(), I observed that the covariate dataset contains 157 rows and 10 columns, the gene expression dataset contains 500 rows and 158 columns, and the gene annotation dataset contains 750 rows and 2 columns. The covariate dataset includes a geoID variable, while the gene annotation dataset contains probe IDs and corresponding gene names. The gene expression column names appear to correspond to the sample IDs (stored in the geoID column) in the covariate dataset. In the next step, I will verify the sample ID matching and inspect the structure of the gene expression data before merging the datasets.

```
# Confirm that the geoID column exists
"geoID" %in% names(covar)
```

```
## [1] TRUE
```

```
# Inspect sample IDs from the covariate file
head(covar$geoID)
```

```
## [1] "GSM2264346" "GSM2264347" "GSM2264348" "GSM2264349" "GSM2264350"
## [6] "GSM2264351"
```

```
# Inspect the gene expression column names
head(names(gene_expr), 10)
```

```
##  [1] "...1"       "GSM2264363" "GSM2264379" "GSM2264394" "GSM2264418"
##  [6] "GSM2264449" "GSM2264452" "GSM2264362" "GSM2264419" "GSM2264416"
```

```
# Count how many covariate sample IDs match
# gene expression column names
sum(covar$geoID %in% names(gene_expr))
```

```
## [1] 157
```

```r
# Check for duplicate sample IDs in the covariate data
sum(duplicated(covar$geoID))
```

```
## [1] 0
```

```r
# STEP 3: Verifying sample IDs

# First checking how many unique sample IDs are in the covariate dataset
length(unique(covar$geoID)) # RESULT: 157 which means that there are no duplicates
```

```
## [1] 157
```

```r
# Now, checking whether the covariate sample IDs are there in gene expression
# column names and how many there are
sum(covar$geoID %in% names(gene_expr)) # RESULT: 157 which means that there are
```

```
## [1] 157
```

```r
# 157 instances of them matching. Since there are 157 rows in covar, corresponding
# to 157 sample IDs, our result confirms that 157 matches are there, i.e., the 157
# sample ids in covar are there in the column names for gene_expr.

# Checking to see if there are any covariate sample IDs that do not match
# a column name in the gene expression data
setdiff(covar$geoID, names(gene_expr)) # RESULT: 0 which means that all covar sample
```

```
## character(0)
```

```r
# IDs have a match in the column names set for gene expression dataset

# Now, checking the reverse, i.e., if there are any column names in the gene expression
# data set that do not match a covariate sample ID
setdiff(names(gene_expr), covar$geoID) # RESULT: yes, there is one column that
```

```
## [1] "...1"
```

```r
# doesn't match.
```

The covariate dataset contains 157 unique sample IDs, and all 157 IDs were found among the column names of the gene expression dataset. The setdiff() check returned no unmatched covariate sample IDs, confirming that every sample in the covariate dataset has a corresponding column in the gene expression dataset. However, the reverse check identified one additional column, named ...1, in the gene expression dataset. We will now look into this column to see what to do with it.

```r
# STEP 4: Inspecting the extra gene expression column ...1

# Checking again the first few rows of the gene expression dataset to see how the
# values stored in column ...1 compare to the values stored in the otehr columns
head(gene_expr[, 1:6])
```

```
## # A tibble: 6 x 6
##      ...1 GSM2264363 GSM2264379 GSM2264394 GSM2264418 GSM2264449
##     <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 8089659       6.18       6.11       6.32       5.91       6.35
## 2 8083594       4.80       4.79       4.80       4.93       4.98
## 3 8035050       9.08       8.90       9.24       8.82       8.84
## 4 8141107       6.78       7.18       7.17       7.45       7.01
## 5 8093624       8.89       9.42       8.96       9.01       9.10
## 6 7969472       6.44       6.44       6.29       6.23       6.06
```

Looking at the first column "...1", we see that it seems to contain ID like values like 8089659. The other columns on the other hand seem to have more measurement like values. Hence, I am thinking that maybe column 1 contains values that will correspond to the probe id values in the gene names dataset. So, this is what I will check now.

```
# STEP 5: Inspecting probe ID column in combination with "...1" column

# Checking again what the column names in the annotation file are
names(gene_names) # RESULT: "PROBEID", "GENE"
```

```
## [1] "PROBEID" "GENE"
```

```
# Viewing some data in the "...1" from the expression dataset again
head(gene_expr$...1) # RESULT: Values like 8089659, 8083594 [7 digit number]
```

```
## [1] 8089659 8083594 8035050 8141107 8093624 7969472
```

```
# Viewing some probe IDs from the gene name dataset
head(gene_names$PROBEID) # RESULT: Values like 8128043, 7962489 [7 digit number]
```

```
## [1] 8128043 7962489 7920271 7900468 8026496 7989725
```

```
# That's interesting. Both return 7 digit numbers. So maybe, they do align. Let's
# check now if any values from the "...1" column appear in the probe id column in
# gene name. But actually, before we do this, we should remind ourselves that there
# are 500 rows for gene expression dataset meaning that we have 500 values in the
# "...1" column. So if we see 500 matches, it means that all the values found a
# match in the probe column

# Check how many expression probe IDs appear in the annotation file
sum(gene_expr$`...1` %in% gene_names$PROBEID)
```

```
## [1] 500
```

The gene expression dataset contains 500 rows and 158 columns, while the gene name dataset contains 750 rows and 2 columns (PROBEID and GENE). All 500 probe IDs in the gene expression dataset were found in the gene name dataset.

This means that the data in the gene expression dataset can be linked to their corresponding names in the gene name using the "...1" column in gene expression and PROBEID column in gene names.

So to recap, all 157 sample IDS in the covariate dataset have matching expression data (sample ids = column names) and the all 500 values in "...1" have a matching probe ids and hence, gene names.

Our next step will now be to select a gene that we will focus our project on.

```r
# STEP 6: Exploring available genes

# Check how many distinct gene names are present
length(unique(gene_names$GENE)) # Result: 750 so 750 unique gene names.
```

```
## [1] 750
```

```r
# Lets look at some gene names
head(gene_names$GENE, 10)
```

```
##  [1] "CNR1"      "PLEKHA8P1" "S100A4"    "NFYC"      "LINC00661" "PDCD7"
##  [7] "TIFA"      "SLC68A1"   "UNC13A"    "SOCS5"
```

```r
# Also, I am wondering that since there are 500 probe values in column ...1 and
# there are 750 PROBEID/gene names, if there are duplicates for gene names.

# So here's lets see if there are any duplicates and then also how many unique gene
# names exist in the gene name file

# Checking for duplicates for gene names
any(duplicated(gene_names$GENE)) # RESULT: FALSE which means no duplicates
```

```
## [1] FALSE
```

```r
# Counting how many unique gene names are present
length(unique(gene_names$GENE)) # RESULT: There are 750 gene names (matches what
```

```
## [1] 750
```

```r
# we expect)
```

```r
# STEP 7: Checking whether a gene is present in both the gene name dataset and
# gene expression dataset

# Finding the probe ID corresponding to the gene NFYC
gene_names[gene_names$GENE == "NFYC", ] # RESULT: PROBEID = 7900468
```

```
## # A tibble: 1 x 2
##   PROBEID GENE
##     <dbl> <chr>
## 1 7900468 NFYC
```

```r
# Checking if this probe ID is present in the gene expression dataset
gene_names$PROBEID[gene_names$GENE == "NFYC"] %in% gene_expr$`...1` # RESULT: TRUE!
```

```
## [1] TRUE
```

```r
# So the gene that we selected is NFYC. The reason I picked this gene was because
# it sounded like NY and I thought it was pretty cool that it did not have a digit
# in the gene name
```

The gene we will be working with is called NFYC and has a PROBEID of 7900468.

Now that we have chosen our gene, we will work on merging the data into one sample dataset that we can use for our plotting.

```r
# STEP 8: We will begin by extracting the gene expression values for NFYC

# Find the row containing the NFYC probe ID (=7900468) in the gene expression
# dataset and then storing it. Just as a reminder, the probe id in gene names
# corresponds to the values in the ...1 column in gene expression dataset.

nfycRow <- gene_expr[gene_expr$...1 == 7900468,]

# Checking the dimensions of the extracted row [we expect it to be 1 x 158 because
# we are picking one row and we want all the columns for this row and since gene
# expression has 158 columns, we hence expect 158 columns]
dim(nfycRow) # We have 1 x 158 as our dimensions which matches what we
```

```
## [1]   1 158
```

```r
# expect so success!

# Taking a look at this row
#head(nfycRow) # Looks good, so success!
```

Now for plotting purposes where we will be picking 3 covariates for our 3 plots, we want to reshape the nfyc expression row data such that each samples value is now stored under a new column with all the value names stored in the rows instead of columns such that each row corresponds to a sample.

```r
# STEP 9: Reshaping the NFYC gene expression data.

# We will begin by removing the probe ID column as we have now selected the data
# we will be focusing on. We will reshape the data and covert the wide data to
# long data.

nfycExpression <- nfycRow %>%
  pivot_longer(cols = -`...1`,
               names_to = "geoID",
               values_to = "NFYC_Expression") %>%
  select(geoID, NFYC_Expression)


# Looking at the resulting dataset
#head(nfycExpression)

# Checking for the number of samples (since there are 157 samples, we expect
# this to return 157)
nrow(nfycExpression) # RESULT: 157! so that means that the dataset is good to go
```

```
## [1] 157
```

Now that we are done reshaping the NFYC expression data, we now have a dataset with 157 rows with each row corresponding to a sample and containing both the sample ID (stored in geoID) and NFYC expression values (stored in NFYC_Expression).

We will now do our final step for making our own sample dataset by merging this expression data with the covariates data on the geoID column which is present in both datasets. (we in fact called our sample ID column as geoID because this is what it is called in the covariates data set.)

Once, we do this, we will have a NFYC dataset containing both expression data and covariate data.

```
# STEP 10: Joining the NFYC expression dataset with covariates dataset

# Joining the NFYC gene expression data to its corresponding covariates

nfycData <- covar %>% inner_join(nfycExpression, by="geoID")

# Quickly checking the dimensions of the merged dataset (we expect it to be
# 157 x 10+1, i.e., 157 rows since we have 157 samples and then 11 columns since
# there are 10 covariate columns and 1 expression column and since we are joining
# the two data sets so as to have covariate and expression data for the 157 samples
# specifically for this gene, we expect the merged dataset to have dimensions
# 157 x 11)
dim(nfycData) # RESULT: 157 X 11! This means our merge seems successful. Let's
```

```
## [1] 157  11
```

```
# also take a look at the merged dataset itself to confirm this.

# Looking at the merged dataset
#head(nfycData) # This looks right, with all the covariate data present for the
# different samples and the expression column added to the end.

# Verify that each sample appears only once
sum(duplicated(nfycData$geoID)) # RESULT: 0. this means that there are no sample
```

```
## [1] 0
```

```
# duplicates which is good and what we expect
```

Now that we have merged our data, when we inspect our column values further, we can notice that it can benefit from some cleaning up. For instance, instead of having age_years: 31.06639288 in mat_age_yrs, we can have 31.06639288 as this will allow the data to look cleaner and neater. It will also make it easier to read and analyse.

```
# STEP 11: Cleaning the data by checking for any missing values, duplicates, etc.

# Checking the no. of missing values in each column
colSums(is.na(nfycData)) # RESULT: 0 for all columns which means that there are
```

```
##                   subjectID                       geoID
```

```
##                             0                             0
##                 mat_age_yrs           preg_preeclampsia
##                             0                             0
##         delivery_gestAge_wks sampleCollection_gestAge_wks
##                             0                             0
##             batch_Microarray                       mat_BMI
##                             0                             0
##                      mat_vitD                   mat_asthma
##                             0                             0
##              NFYC_Expression
##                             0
```

```
# no NAs present in our dataset.

# Checking for duplicated rows
sum(duplicated(nfycData)) # RESULT: 0 which means that there are no duplicated rows
```

```
## [1] 0
```

Now that we have checked that there are no NAs and duplciated rows, we will now focus on cleaning up the column values.

```
# STEP 12: Cleaning the data specifically cleaning up the columns

# Taking a look at teh data so that we know what to put down for our mutate()
#head(nfycData)

# Creating a copy so our new nfyc dataset remains unchanged
cleanNFYCData <- nfycData

# Removing descriptive labels (using gsub) and converting to numerical variables
# where appropriate (using as.numeric())
cleanNFYCData <- cleanNFYCData %>%
  mutate(
    # Removing both descriptive labels and converting to numeric (so it is treated as such)
    mat_age_yrs = as.numeric(gsub("age_years: ",
                                  "", mat_age_yrs)),
    delivery_gestAge_wks = as.numeric(gsub("gestation_weeks: ",
                                           "", delivery_gestAge_wks)),
    sampleCollection_gestAge_wks = as.numeric(gsub("blood_draw_at_gestation_week: ",
                                                   "", sampleCollection_gestAge_wks)),
    mat_BMI = as.numeric(gsub("bmi_kg/m2: ",
                              "", mat_BMI)),
    # because parentheses are not treated as literal characters and are treated
    # specially in regular expressions, we add \\ to them so that R treats them
    # as being literal parentheses.
    mat_vitD = as.numeric(gsub("vitamind_25\\(0h\\)_ng/ml_whole_blood_baseline: ",
                               "", mat_vitD)),

    # Only removing descriptive labels
    preg_preeclampsia = gsub("pregnancy_condition: ","", preg_preeclampsia),
    batch_Microarray = gsub("microarray_hybridization_date: ", "", batch_Microarray),
```

```
    # Removing descriptive labeling and converting 0 to Healthy and 1 to Asthmatic
    mat_asthma = gsub("asthma_0=healthy_1=asthmatic: ", "", mat_asthma),
    mat_asthma = ifelse(mat_asthma=="0", "Healthy", "Asthmatic"))


# Taking a look at our data to see if it looks good.
#head(cleanNFYCData) # RESULT: Looks good! Looks much neater and cleaner.
```

Now that we have cleaned up within the columns, we will do another check to see if there is any missing data, i.e., if anything became NA. If we did it correctly, we will see no NA values because our NFYC dataset didn't have any NA values to begin with.

```
# STEP 13: Cleaning the data by checking for any missing values

# Checking the no. of missing values in each column
colSums(is.na(cleanNFYCData)) # RESULT: 0 for all columns which means that there are
```

```
##                    subjectID                        geoID
##                            0                            0
##                 mat_age_yrs           preg_preeclampsia
##                            0                            0
##         delivery_gestAge_wks sampleCollection_gestAge_wks
##                            0                            0
##             batch_Microarray                      mat_BMI
##                            0                            0
##                     mat_vitD                   mat_asthma
##                            0                            0
##              NFYC_Expression
##                            0
```

```
# no NAs present in our dataset.
```

Great, there is no missing data in our cleaned dataset so our dataset is now ready to be used for plotting.

In order to start with the plotting, we have to pick the which variables we will be using to make the plots. According to the assignment, we have to select: - 1 gene (which we picked earlier to be NFYC) - 1 continuous covariate (we can use maternal age to be this as there is a range of ages present) - 2 categorical covariates - 1 existing categorical covariate (we can use pregnancy condition to be this wherein samples are divided into normal and preeclampsia groups) - 1 newly converted categorical covariate (i.e., continuous -> categorical)... to pick this one, we need to choose a continuous variable that can be divided into meaningful categories.

When looking at the cleaned dataset, I was looking at the vitamin D category which is continuous. I was curious to explore if Vitamin D levels had an association, if any, with gene expression levels and pregnancy preeclampsia condition status.

So, we will be looking at this variable properly and see how we divide it into two categories. Note that when we are comparing groups, sample sizes should be kept in mind because very small groups can make the distributions more difficult to interpret and although groups do not need to be exactly equal in size, having a reasonable number of samples in each group can make the comparison more informative.

```
# Checking vitamin D levels data
summary(cleanNFYCData$mat_vitD) # Range from ~5 to ~81 so pretty wide range range
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    5.11   13.40   20.40   22.98   29.50   80.80
```

```
# Creating vitamin D groups using 20 ng/mL as the cutoff.
# I selected this cutoff after researching commonly used reference thresholds.
# Reference: https://www.ncbi.nlm.nih.gov/books/NBK532266/
cleanNFYCData <- cleanNFYCData %>%
  mutate(mat_vitD_group = factor(ifelse(mat_vitD < 20,
                                        "Below 20 ng/mL",
                                        "20 ng/mL or above"),
                                 levels = c("Below 20 ng/mL",
                                            "20 ng/mL or above")))

# Checking the number of samples in each group
table(cleanNFYCData$mat_vitD_group) # Pretty evenly split so sample group sizes
```

```
##
##    Below 20 ng/mL 20 ng/mL or above
##                78                79
```

```
                                    # are pretty similar
```

Since these groups have similar sample sizes, I will use vitamin D as the second categorical variable. This will allow us to compare NFYC gene expression across vitamin D groups alongside pregnancy condition. Note that here I am using the 20 ng/mL cutoff to organise the data for this analysis and it should not be interpreted as a definitive diagnosis of an individual's vitamin D status.

. . . . . .

So to summarize now, the variables we will use for our analysis are as follows: - Gene of Interest: NFYC gene expression ("NFYC_Expression") - Continous Covariate: Maternal age ("mat_age_yrs") - First Categorical Covariate: Pregnancy condition ("preg_preeclampsia") - Second Categorical Covariate: Maternal Vitamin D group ("mat_vitD_group"; from "mat_vitD")

We will now use these variables to create three plots. - Histogram (to visualise the distribution of NFYC gene expression across all samples). This will help us understand how the expression values are distributed and whether most samples have similar expression levels or whether there is a wider spread in the data. - Scatterplot (to explore the relationship between maternal age and NFYC expression) - Boxplot (to compare NFYC expression across pregnancy preeclampsia condition and Vitamin D groups)

Let's begin with the histogram! (I am first going to create my custom theme)

```
# Here to create my own custom theme, I am making a dark background theme with
# the axes and text in white. The legend is placed at the bottom. We will also
# remove all the extra border and grid lines to make it look cleaner.
newBlackTheme <- theme(

  # Remove all the extra borders and grid lines
  panel.border = element_blank(),     # remove the borders
  panel.grid.major = element_blank(), # removes the major grid lines
  panel.grid.minor = element_blank(), # removes the minor grid lines

  # Sets the axis color and text to white
  axis.line = element_line(color="white"),               # sets axis line color to white
  axis.text = element_text(colour = "white", size = 14), # sets axis text to white
```

```
  axis.ticks = element_line(colour = "white"),           # sets axis tick to white

  # Sets plot background to black
  plot.background = element_rect(fill = "black"), # this sets background color to black
  panel.background = element_blank(),             # this removes the panel background

  # This centers the plot title
  plot.title = element_text(hjust = 0.5, size = 15),

  # Sets text color to white
  text = element_text(colour = "white", size = 14),

  # Sets legend color as black
  legend.key = element_rect(fill = "black"), # this sets legend key background to black
  legend.background = element_rect(fill = "black"), # this sets legend background to black

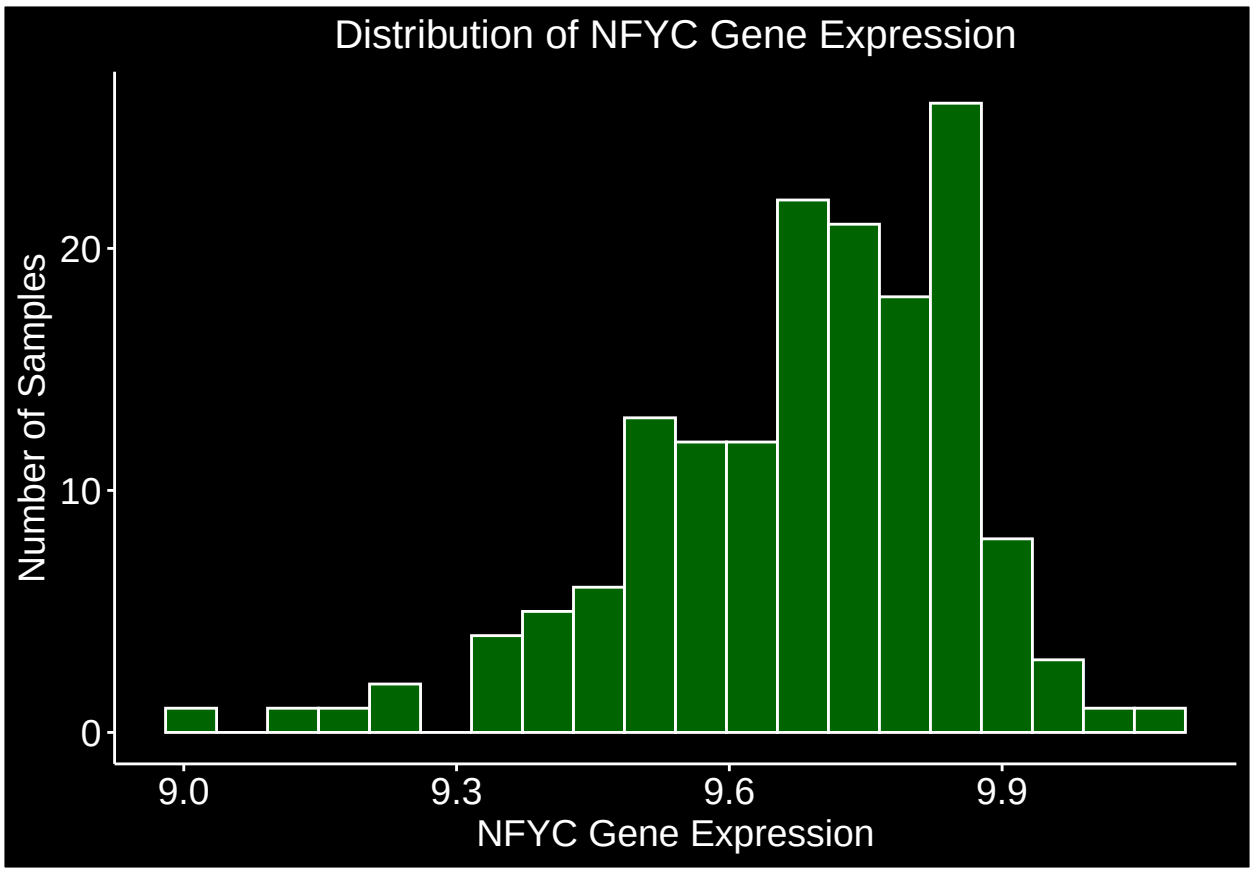
  # Sets legend text color as white
  legend.text = element_text(colour = "white", size = 14),
  legend.title = element_text(colour = "white", size = 14),

  # Sets legend position to bottom
  legend.position = "bottom")
```

```
# STEP 14: PLOTTING (PLOT 1): Creating a histogram of NFYC gene expression

library(ggplot2)

ggplot(cleanNFYCData, aes(x = NFYC_Expression)) +
  geom_histogram(bins = 20, fill = "#006400", color = "white") +
  labs(title = "Distribution of NFYC Gene Expression",
       x = "NFYC Gene Expression",
       y = "Number of Samples") +
  newBlackTheme
```




```
# Saving this:
ggsave("Submission1_NFYC_histogram.pdf", width = 8, height = 6)
```

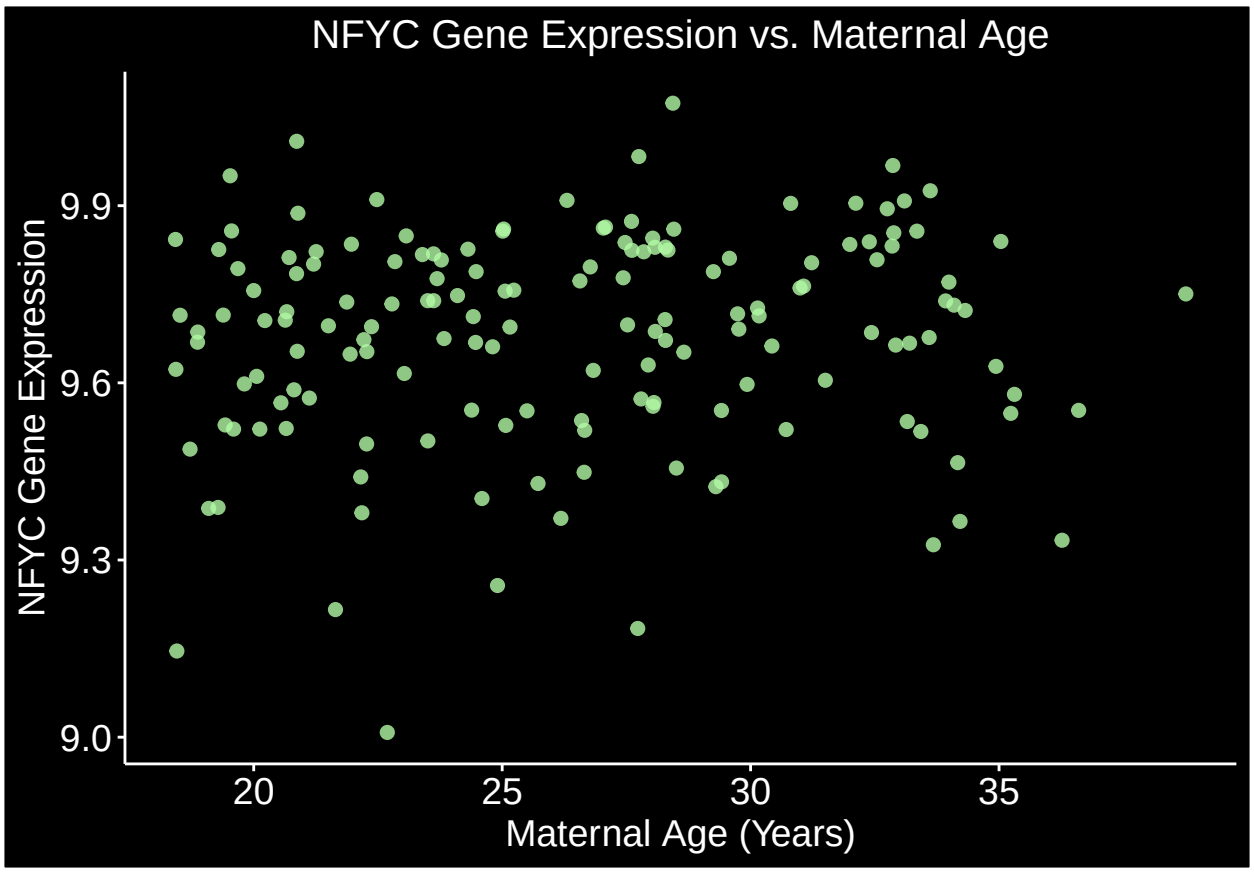
After creating the histogram, we can see how NFYC gene expression is distributed across our 157 samples. Most of the samples have NFYC expression values between approximately 9.5 and 9.9, with the highest concentration of samples appearing around 9.7 to 9.9. There are also a few samples with lower expression values, particularly around 9.0 to 9.3, as well as a small number with values above 10.0. Overall, the distribution appears to have most of its values concentrated toward the higher end, with a longer tail extending toward the lower values.

This gives us a better understanding of the variation in NFYC gene expression across the samples. However, the histogram only shows the distribution of gene expression on its own, so it does not tell us whether expression levels are related to any of the maternal characteristics (like age, etc) in our dataset.

To explore this further, we will now create a scatterplot to investigate whether maternal age is associated with NFYC gene expression.

```
# STEP 15: PLOTTING (PLOT 2): Scatterplot of maternal age and NFYC gene expression

ggplot(cleanNFYCData, aes(x = mat_age_yrs, y = NFYC_Expression)) +
  geom_point(color = "#B2FBA5", size = 2, alpha = 0.8) +
  labs(title = "NFYC Gene Expression vs. Maternal Age",
       x = "Maternal Age (Years)",
       y = "NFYC Gene Expression") +
  newBlackTheme
```




```
# Saving this:
ggsave("Submission1_NFYC_scatterplot.pdf", width = 8, height = 6)
```

From the scatterplot, we can see that NFYC gene expression values vary quite a bit across the different maternal ages, with most samples having expression values that lie between ~9.6 and 9.9. We can also see that there are also a few samples with considerably lower expression values.

Looking at the overall pattern, there does not appear to be a strong linear relationship between maternal age and NFYC gene expression. The expression values are quite spread out across the age range, and mothers of similar ages can have different levels of NFYC expression. However, this is only a visual observation, so we cannot conclude concretely from the plot alone whether maternal age is associated with NFYC gene expression. Further statistical analysis would be beneficial in investigating this relationship more closely.

For our final plot, we will use a boxplot to compare NFYC gene expression across the two categorical variables we selected: pregnancy condition (Normal or Preeclampsia) and vitamin D group (Below 20 ng/mL or 20 ng/mL or above).

```
# STEP 16: PLOTTING (PLOT 3): NFYC expression by pregnancy condition and
                            # Vitamin D group

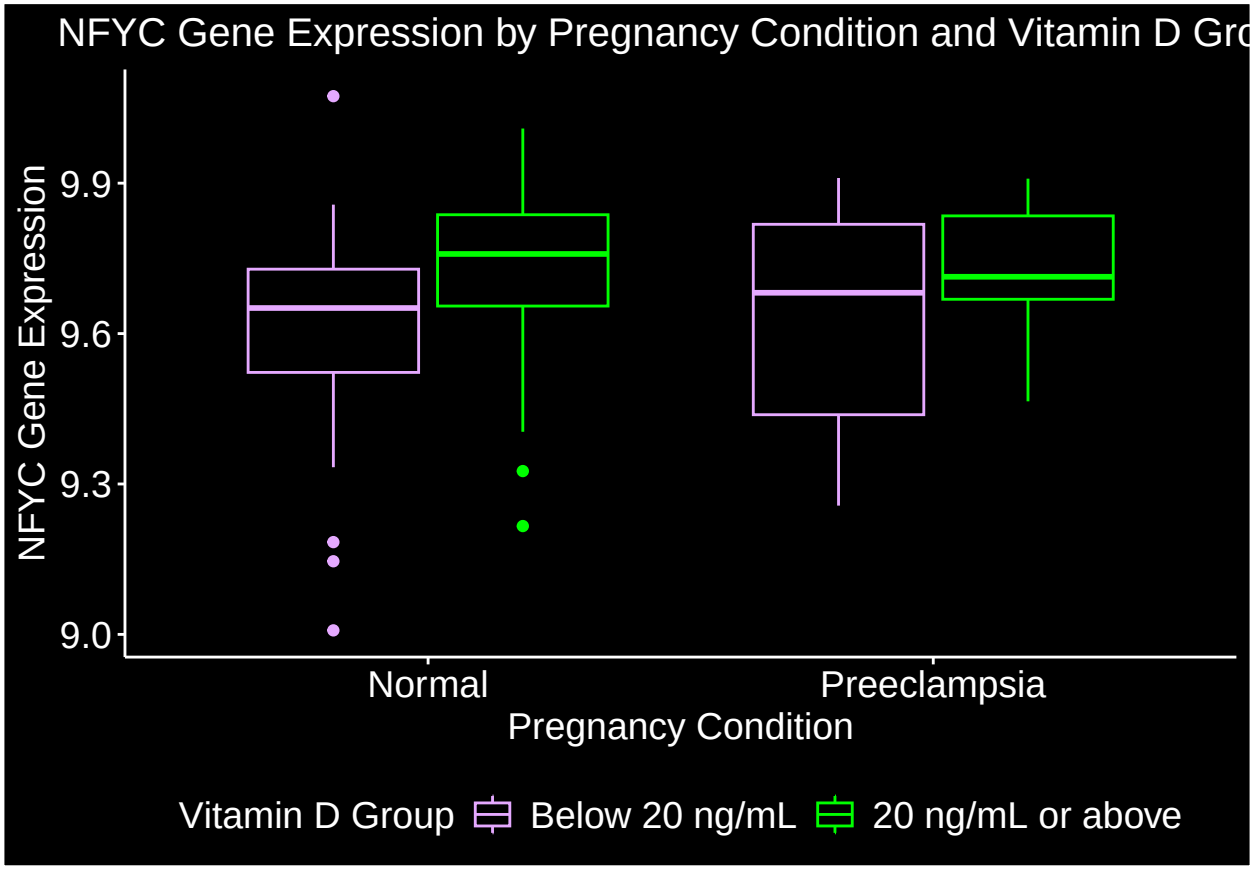
vitD_Colors <- c("#E5A9FF", "#00FF00")

ggplot(cleanNFYCData, aes(x = preg_preeclampsia,
                          y = NFYC_Expression,
                          color = mat_vitD_group)) +
  geom_boxplot(fill="black") +
```

```
# Defining colors for the plot
scale_color_manual(values = vitD_Colors) +

labs(title = "NFYC Gene Expression by Pregnancy Condition and Vitamin D Group",
     x = "Pregnancy Condition",
     y = "NFYC Gene Expression",
     color = "Vitamin D Group") +

newBlackTheme
```




```
# Saving this
ggsave("Submission1_NFYC_boxplot.pdf", width = 8, height = 6)
```

From the boxplot, specifically zoning in on the medians, we can see that among participants with normal pregnancies, those with vitamin D levels of 20 ng/mL or above appear to have higher median NFYC expression (~9.8) compared to those with vitamin D levels below 20 ng/mL (~9.7). However, among participants with preeclampsia, the median expression values are more similar between the two vitamin D groups, with both seeming to be around 9.7 albeit the group with vitamin D levels of 20 ng/mL or above appearing to be relatively slightly higher.

We also observe differences in the spread of the expression values. Among participants with preeclampsia, the group with vitamin D levels below 20 ng/mL appears to have a wider interquartile range, suggesting greater variability in the middle 50% of NFYC expression values. In contrast, the normal-pregnancy groups appear to have narrower interquartile ranges, although some potential outliers are present at the lower end for the group with vitamin D levels of 20 ng/mL or above and on both the ends for the group with vitamin D levels below 20 ng/mL.

Another interesting observation is that among participants with vitamin D levels below 20 ng/mL, the median NFYC gene expression appears to be slightly lower in the normal-pregnancy group than in the preeclampsia group. The reverse is then seen in the other group, where among participants with vitamin D levels 20 ng/mL or above, the median NFYC gene expression appears to be slightly higher in the normal-pregnancy group than in the preeclampsia group. These are interesting patterns because the differences in median NFYC gene expression between the vitamin D groups appear to vary depending on pregnancy status. This raises the question of whether the relationship between vitamin D levels and NFYC gene expression differs depending on pregnancy condition. However, we do have to note that the difference in median values is relatively small, and further statistical testing would be needed to determine whether this pattern is meaningful or simply due to variation within the sample.

Looking at the overall pattern, the boxplot suggests that the differences in median NFYC expression between the vitamin D groups may be more noticeable among participants with normal pregnancies than among those with preeclampsia. However, this is only a visual observation, so we cannot conclude concretely from the plot alone whether differences are statistically significant or whether vitamin D levels are, in fact, associated with NFYC expression. Further statistical analysis would be beneficial in investigating this relationship more closely.

And that is it, for now (i.e., for Submision 1)!